Kavi

Engineering and Research Specification

Arnav123-s

7 September 2026

Abstract

Kavi investigates learning reusable operations as executable circuit structure. Teaching examples diagnose candidate procedures; a wrong result drives repair of the shared computation. The retained model consists of gates, connections and operations, while temporary signals and teaching episodes are outside its deployed memory. The broader objective is an adaptive computational graph that acquires useful structure and preserves required earlier behavior.

A bounded structural learner is now implemented. It acquires a binary-addition transition from whole-operand examples using AND, XOR and NOT gates under a supplied streaming executor. Three declared seeds produced the same five-gate, 321-byte circuit. Each passed 127 independent four-bit pairs, all 65,536 pairs in an eight-bit audit, and 190 longer-input cases up to 1,024 bits, with zero audit regressions. The full run took 5.281 seconds on one CPU process. The audit includes selection examples; the reserved and longer-input banks are separate final tests.

Two subsequent live trials acquired subtraction and small programs over earlier operations. A 113-byte acquired call arrangement enabled powers and factorials to pass all declared follow-up tests. A retained library combines 13 compatible acquired procedures in 2,238 bytes. The larger vocabulary also made one task time out. These results depend on supplied types, iteration semantics and search; general abstraction invention, shared repair, prose interpretation and advanced subject capability remain open. Earlier symbolic and 66,880-parameter recurrent experiments remain separate systems. The mathematical basis connects graph rewriting, finite-state transducers and typed program induction.

Structural learner baseline
`0453052f83d9a02a126b37fd15900c092ce2ed0b`

Contents

1 Research objective and design requirements 3

2 Implemented systems and evidence boundaries 4

3 Current text model mathematics 6

4 Experimental findings 8

5 Closest mathematical relatives 12

6 A formal specification for the intended learner 13

7 Capacity and the meaning of compression 16

8 Continual learning and preservation 17

9 Physical dynamics and mathematical corrections 18

10 Machine learning components and their roles 20

11 Software architecture 21

12 Curriculum and source engineering 23

13 Experiments that can resolve the main questions 24

14 Engineering and operation 26

15 Development milestones 27

16 Prospects and attainable levels 29

17 Technical study sequence 30

Appendix A Implemented component inventory 31

Appendix B Additional design hypotheses 33

Appendix C Evidence boundaries 35

Appendix D Research references 36

Appendix E Repository evidence 38

1 Research objective and design requirements

1.1 The learned object

The central proposal is that knowledge resides in the configuration of executable paths. A path is an ordered computation with explicit input types, operations, branches and outputs. Multiple tasks may share a subcomputation. An arithmetic expression and a sentence may use the same character ingress while requiring different interpretations of the same symbol. Context determines the applicable contract before their computations diverge.

This representation can support several forms of learning. A missing connection can be added; an incorrect branch condition can be repaired; repeated computation can become a shared procedure; a sequence of operations can be replaced by an equivalent shorter implementation. The identity of the learner is its continuing body of acquired behavior and structure, rather than an immutable parameter vector.

The research question is whether these operations can be learned economically and generalize beyond their teaching examples. A graph containing a hand-written solution is an implementation of that solution. Evidence for a learning mechanism requires the graph to be acquired through a declared teaching process, with the target solution withheld from the learner.

1.2 Required behavior

Requirement	Operational meaning	Evidence needed
Learn from correction	A verified target changes the learner and improves a later response	Before and after results, plus new examples
Reuse existing computation	A later task invokes a previously acquired subprocedure	Executed call trace and transfer comparison
Retain earlier abilities	Previously correct cases remain correct after learning	Per-case transitions and independent confirmation
Continue adapting	Internal state and structure remain editable	Recorded changes with retained behavior
Limit growth	Learning stays within declared persistent and temporary budgets	Serialized bytes, peak memory and search cost
Compress meaningfully	A smaller representation preserves specified behavior	Equivalence proof or measured distortion
Support context	Identical surface symbols can participate in different tasks	Context-sensitive tests without answer leakage
Remain inspectable	Traces show actual executed operations and decisions	Trace-to-execution consistency

Developmental stages organize prerequisites and teaching difficulty. A stage score is a progress measure. Resetting its baseline does not reset storage, remove earlier knowledge, or create capacity. The useful interpretation of growth and consolidation is that a newly learned abstraction can replace several redundant descriptions.

Backpropagation is an available comparator and is used by the current text core. The program-learning objective does not require gradients, although a learned proposal policy could use them. Avoiding a particular optimizer is secondary to specifying what changes, how a correction selects that change, and why the resulting procedure works.

1.3 Scope

The learner core is the research subject. Teachers, source loaders, evaluators, experiment runners and terminal views support its development. They must be accounted for because they supply data, supervision, search and computation, but their capabilities cannot be attributed to the core.

The structural circuit experiment is implemented and was run under its declared finite configuration following project authorization. The earlier recurrent curriculum was not resumed. Broader curriculum restarts, hardware-policy changes and autonomous host source-code modification remain separate execution decisions.

2 Implemented systems and evidence boundaries

2.1 Repository structure

The earlier baseline contains 52 package modules and 12,193 lines. Four circuit-learning and six procedure-library modules bring the current inventory to 62. The original 138 tests passed after relocation, 153 passed after circuit implementation, and 169 passed after the procedure and scaling extensions under Python 3.13.5 with PyTorch 2.6.0. The first gate trial used Python 3.12.14; the library trials used Python 3.13.5 and the standard library. Unit tests establish specified software behavior, not language understanding or scientific competence.

Three representations must remain distinct throughout development.

Representation	State that changes	Current status
Symbolic pathway circuit	Prototype centers, support counts, arithmetic coefficients, typed route contracts and adapters	Implemented and tested on restricted tasks
Recurrent text circuit	Byte embeddings, output projection, routing coefficients, phases and memory gates	Implemented; measured short-sequence learning
Executable circuit and program graph	Gate operations, connections and small programs calling acquired procedures	Transition acquisition and bounded typed program learning implemented; broader control and abstraction invention remain proposed

The physics-native design is another proposed core, with continuous state and an energy function. It is not an explanation of what the current recurrent code secretly computes. Its equations need their own implementation and validation.

2.2 Symbolic pathway circuit

The early arithmetic fabric uses typed signals, compatible routes and a small learned numeric readout. Dijkstra search selects a minimum declared-cost route in an existing graph. This solves a routing problem after the available operations and costs have been specified; it does not discover the mathematical solution to an arbitrary problem.

The unified circuit adds compact glyph and script prototypes, expression-versus-relation classification, arithmetic transforms, jump adapters and typed composition. A composition request is already represented as an operator tree. The curriculum supplies contracts connecting operators such as addition, classification, equality and selection to their target paths. The runtime can execute unfamiliar combinations of these supplied contracts within finite depth and node limits.

This is useful infrastructure for a future program learner. It establishes typed execution, state persistence and compositional interfaces. It does not demonstrate acquisition of the operators from unstructured text. In particular, `arithmetic_target_weights` supplies the exact coefficients for addition and subtraction, while

`composition_units` supplies structural bindings. These are substantive forms of supervision. Symbolic core and composition curriculum.

The Unicode prototype also has a precise boundary. It accepts a single Unicode scalar and classifies declared examples using compact code-point features and centroids. It does not receive glyph images. A script label does not identify a language, and success on a small generated set does not establish coverage of the Unicode standard.

2.3 Recurrent text circuit

The text core receives UTF-8 bytes and predicts subsequent bytes. It is a classical sparse recurrent neural network whose complex values are represented by pairs of real coordinates. Ordinary training updates continuous parameters through time. Its default source-index graph remains fixed except in explicitly implemented structural experiments.

The model starts without pretrained weights. That condition does not remove its inductive biases: the byte representation, topology, update equations, optimizer, prompts, curriculum and targets are all engineered. The relevant question is how much the learner acquires beyond these supplied structures. Text core.

2.4 Supporting systems

The teacher assembles source passages, exercises, corrections and rehearsal. It selects a lesson order and decides when to give harder questions. The learner supplies predictions. The evaluator compares those predictions with targets. Run management saves model and optimizer state, source fingerprints, events and candidate results.

The earlier symbolic checkpoint and the byte model are separate states. Preserving a symbolic foundation file does not transfer its abilities into the recurrent weights. Likewise, an exact answer produced by a symbolic evaluator is a capability of that evaluator unless the learner independently produces the same answer.

2.5 Discrete structural learner

The new implementation has four modules: `circuit_core` for strict graph data and execution, `circuit_search` for acquisition and counterexamples, `circuit_runtime` for the teacher, controller and final evaluator, and `circuit_cli` for live inspection and queries. The model contains gate choices, connection addresses and output references. Inference requires only that artifact and its interpreter.

The current circuit is a two-state Mealy transducer. Two operand bits and one temporary state bit enter a learned combinational graph; one output bit and the next state leave it. The state starts at zero and the interpreter repeats the graph across input positions, followed by one zero-input frame. Gate selection and wiring are learned. Framing, the state register and the loop are supplied. The graph does not reshape itself during an individual query.

Search enumerates Boolean expressions, compiles shared subexpressions and filters candidate transition pairs using counterexamples. A selected graph must satisfy the entire teaching and protection bank before being installed. A fixed node budget and ranking prefer small structure; no target-specific whole-input lookup is available to the graph. The external search uses truth masks for speed, while deployed inference executes gate nodes.

2.6 Acquired procedure library

Six modules extend the gate learner: `procedure_core`, `procedure_search`, `library_curriculum`, `library_runtime`, `library_cli` and `learning_window`. A strict library stores acquired gate circuits and program trees. Programs can read arguments, use zero or one, call earlier procedures, repeat a binary operation with a fixed step, or fold it over an integer range. The learner chooses names to call, argument arrangements and nesting. It does not discover the supplied instruction semantics.

Values are natural numbers of at most 4,096 bits. Signatures accept one through three inputs, with explicit ordered-input contracts where required. Calls only target earlier definitions, preventing dependency cycles. Execution counts actual calls, gate work and iterations under finite fuel. Each query resets temporary circuit state. Teachers, source records and search caches are absent from deployed inference.

Search enumerates expressions by instruction-tree size and uses recency as a declared prior over acquired calls. It retains one representative per output vector on the teaching bank. That observational pruning is heuristic, not a proved semantic rewrite or a complete search guarantee. A selected program is freshly executed against all teaching targets. Full library selection ends before final evaluation and its canonical artifact is sealed by hash.

The teacher formalizes reviewed De Morgan passages as arithmetic exercises. Scaling, powers, triangular sums and sums of squares are separately authored composition probes. The learner does not interpret prose. Earlier definitions remain immutable during these trials; their retention is append-only preservation. Runtime details are in `docs/PROCEDURE_LIBRARY_RUNTIME.md`.

3 Current text model mathematics

3.1 Representation and parameter accounting

Let the input byte be x_t and the recurrent state be s_t. The default configuration has N = 64 nodes, K = 4 incoming slots per node and H = 2 mixing hops per byte. Each state coordinate has a real and an imaginary component.

$$x_t \in \{0, \dots, 255\}, \qquad s_t \in \mathbb{C}^N.$$

Stored quantity	Shape	Purpose
Byte embedding E	256 by 2N	Encode each input byte
Readout W and bias b	256 by 2N and 256	Predict the next byte
Source indices S	N by K integers	Select each incoming slot's source
Base logits a	N by K	Set routing preferences
Activity gains v	N by K	Condition routing on source magnitude
Conductances g	N by K	Bound transmitted strength
Phases phi	N by K	Rotate two-coordinate signals
Memory gates mu	N	Blend previous state and current input

The source offsets are 0, 1, 3 and 7 modulo N. They are initial graph coordinates, with no demonstrated correspondence to conceptual regions.

$$P = 2(256)(2N) + 256 + 4NK + N = 66{,}880.$$

The embedding and readout matrices contain 65,536 parameters, or 97.99 percent of the total. All parameters occupy 267,520 bytes at float32. The recorded initialized optimizer tensors occupy approximately 535,072 additional bytes. Gradients, activations, graph indices, Python objects, runtime libraries and checkpoints are extra. Counting only the route coefficients would substantially understate the model.

This accounting does not establish that the matrices provide 98 percent of the capability. Attribution requires controlled freezing or replacement experiments. It does establish that the present learned representation is not confined to executable routes.

3.2 Forward computation

Convert the embedding into a complex vector e_t and set alpha_i to the sigmoid of mu_i. The initial state for each byte is

$$z_i^{(0)} = \alpha_i s_{t-1,i} + (1-\alpha_i) e_{t,i}.$$

For each mixing hop, gather the source signals and compute a softmax across the K available slots.

$$u_{ir} = z_{S_{ir}}, \qquad q_{ir} = a_{ir} + \tanh(v_{ir})|u_{ir}|,$$

$$\pi_{ir} = \frac{\exp(q_{ir})}{\sum_{r'} \exp(q_{ir'})}, \qquad m_i = \sum_r \pi_{ir}\sigma(g_{ir}) e^{\mathrm{i}\phi_{ir}} u_{ir}.$$

The hop adds a residual message and an input contribution, then normalizes the result.

$$\tilde{z}_i = z_i + \tfrac{1}{2} m_i + \tfrac{1}{4} e_{t,i}, \qquad z_i^{\mathrm{new}} = \frac{\tilde{z}_i}{\sqrt{1+|\tilde{z}_i|^2}}.$$

After H hops, flatten the real and imaginary coordinates in interleaved node order and compute the output logits. Define vec_R(s) as `(Re s_1, Im s_1, Re s_2, Im s_2, ...)`, matching the tensor layout in the implementation.

$$s_t = z^{(H)}, \qquad \ell_t = W \,\mathrm{vec}_{\mathbb{R}}(s_t) + b.$$

Generation chooses the largest logit, feeds that byte back into the same network, and stops on newline or the output limit. Independent questions start from a reset sequence state. This is deterministic greedy generation for fixed parameters and input.

The softmax routes are numerically sparse in their available connectivity, but every allocated slot is evaluated. A very small gate does not skip its arithmetic. Sparse connectivity, sparse activation and sparse execution are different engineering properties.

3.3 Complex arithmetic and stability

A complex linear map has an exact real-coordinate representation.

$$(A+\mathrm{i}B)(x+\mathrm{i}y) \longleftrightarrow \begin{bmatrix} A & -B \\ B & A \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}.$$

The benefit, if any, comes from the imposed structure, parameter sharing and optimization behavior. It must be compared with real-valued alternatives. The complex-network literature reports task-dependent results, including settings where real networks perform better. Deep Complex Networks.

The normalization ensures a bounded magnitude after each hop for finite inputs.

$$|z_i^{\mathrm{new}}|^2 = \frac{|\tilde{z}_i|^2}{1+|\tilde{z}_i|^2} < 1.$$

It does not establish contraction of the complete recurrent mapping, adequate memory, stable gradients or correct answers. Those properties depend on the Jacobian of the full recurrence, including activity-dependent routing. A bounded state can still lose the information required to distinguish first, middle and last positions.

3.4 Objective and learning

Answer-focused learning combines a prompt prefix with a verified answer and newline. Only target answer bytes contribute directly to the loss; the prefix influences them through recurrent state. Each example's loss is normalized by its answer length, and examples receive equal weight.

$$L(\theta) = -\frac{1}{B}\sum_{b=1}^{B}\frac{1}{|T_b|}\sum_{t\in T_b}\log p_\theta(y_{b,t} \mid p_b, y_{b,<t}).$$

Backpropagation through time computes gradients. Their global norm is clipped to one, and Adam updates the parameters. Training uses supplied earlier answer bytes when predicting later answer bytes. Evaluation generates its own bytes. A low training loss therefore does not guarantee a correct complete generated answer.

The `learn` method uses truncated sequence segments; `learn_answers` carries gradients through the full bounded example. Four independent examples can be processed together or in smaller microbatches if their gradients are accumulated before one optimizer step. Four separate Adam steps define a different algorithm.

3.5 Computational cost

For a sequence of length T, the sparse mixing work scales approximately with T H N K. The dense readout adds work proportional to T N V, where V is the byte vocabulary size. Automatic differentiation also retains intermediate computations across training time. A sparse recurrent graph can therefore have a small parameter array while still requiring substantial execution and runtime memory.

The current implementation loops through bytes in Python and performs many small tensor operations. On this device, dispatch overhead and memory traffic may dominate an individual arithmetic operation. Profiling is needed before proposing GPU execution, more nodes, additional hops or low precision. Smaller parameter growth is not evidence of a smaller runtime cost.

4 Experimental findings

4.1 What the measurements establish

The experiments provide evidence that the text model can change its predictions in response to supervised corrections. Familiar symbol copying can become accurate. The current learning rule also exhibits interference: teaching one set of answers can make previously correct answers wrong. Generalization to longer strings is substantially weaker than performance on short strings.

All figures below are recorded experiments from 4 September 2026. Their local JSON reports were checked against the published experiment summaries. The experiments use different sealed question banks; their absolute percentages are not one continuous benchmark curve.

Each main bank contains 419 questions: 96 three-symbol operations, 96 four-symbol operations, 64 five-symbol transfer cases, 64 mixed-script transfer cases, 24 familiar two-symbol retention cases and 75 single-symbol retention cases. The primary score pools the three- and four-symbol groups, giving 192 cases. Copy, join, first and last operations are balanced within each non-retention group. Questions can share strings across operations, so they are correlated observations.

Experiment	Comparison	Independent result	Interpretation
Teaching schedules	Mixed and mistake-focused teaching from the same checkpoint	Final primary means 42.01 and 43.23 percent; baseline 37.50 percent	Some short-task improvement; no decisive recipe winner

Experiment	Comparison	Independent result	Interpretation
Route changes	Standard, damping, rewiring and split growth	Damped finalist 42.36 percent on the final primary bank	Extra route mechanisms did not establish reliable superiority
Eight repair connections	Ordinary, joint repair and projected repair	Final primary means 41.49, 42.36 and 42.19 percent	Small gains, with previously correct answers still lost
Verified consolidation	Select a small fraction of a previous update	All 196 guard successes retained; two previously correct final cases broken	Finite guard preservation did not generalize universally
Forward repair	Reuse eight repairs or add one jump	Identical final correctness for paired methods in all three seeds	The extra jump provided no measured confirmation benefit

Sources: teaching and pathway comparison, small repairs, consolidation, and forward repair.

The teaching/pathway comparison trained 27 candidates for 360 updates each, totaling 9,720 updates across seeds 53101 to 53103. Its final five-symbol means were only 10.42%, 10.94% and 9.38% for mixed, mistake-focused and damped finalists, compared with 3.13% at baseline. Mixed-script means fell from 43.75% to approximately 32% to 34%. Single-symbol retention stayed at 75/75.

The small-repair comparison trained nine candidates for 180 updates across seeds 53121 to 53123. Its five-symbol mean remained 5.73%. The forward comparison ran six 120-update continuations across seeds 64121 to 64123. These seeds vary continuation behavior from shared starting models; they are not independent complete training runs.

4.2 Retention must be measured per case

In the verified consolidation experiment, final primary accuracy increased from 63/192 to 69/192. Across all 419 final questions, eight wrong answers became correct and two correct answers became wrong. A net improvement of six answers therefore coexisted with forgetting.

The forward experiment began from the latest 80,159-update repair candidate and continued for 120 updates. The older and latest checkpoints were correct on 227 and 228 final cases respectively. Their union contained 258 successes. The starting latest model missed 30 of that union. Final candidates answered 231, 232 and 231 cases correctly, but missed 33, 31 and 32 union cases. More total correct answers did not satisfy the requirement to preserve both generations' successes.

For each case, store the parent and candidate correctness flags. Report gains and losses separately. A mean score or a pooled total cannot express the desired preservation property.

4.3 Resource findings

Eight repair connections add 56 float32 parameters, only 224 parameter bytes. In the reported comparison, optimizer-training time for 180 updates increased from about 23.85 seconds to 34.28 seconds with joint repairs and 34.83 seconds with projection. These additions increased training time by roughly 44 to 46 percent despite increasing parameter count by about 0.084 percent.

Recorded peak process working sets were approximately 482 to 497 MiB across the principal comparisons. These observations include runtime allocations and do not represent the whole device's peak memory or energy use. CPU-temperature telemetry was unavailable in those experiments. The parameter array alone was roughly 0.255 MiB.

4.4 Interpretation and unresolved causes

The evidence is consistent with several explanations: inadequate sequence representation, weak command retention, limited optimization, curriculum imbalance, small capacity and interference across shared parameters. The existing comparisons do not isolate one universal cause.

The current model receives byte sequences. Mistakes on visually similar characters are not evidence of visual recognition. Repeatedly answering a familiar prompt correctly is evidence about that prompt; it does not establish a newly acquired general operation. Original textbook exposure also does not establish comprehension unless the model succeeds on independently constructed questions that require the relevant concepts.

The existing results justify continuing a controlled research program. They do not justify describing the learner as proficient in English, broadly multilingual, at university level, or capable of autonomous scientific discovery.

4.5 Structural circuit trial

The first declared run used source revision `7908156e0241a9df4293e56231660fb234ca44de` and seeds 7, 19 and 31. Each seed first learned from the 81 four-bit pairs without carries, with the next-state output restricted to zero. Repair enabled the supplied state register's transition and used 48 carry cases. The foundation domain constrained repair. The remaining 127 four-bit pairs were excluded from selection.

Seed	Gates / model bytes	Unseen 4-bit pairs	Exhaustive 8-bit pairs	Longer inputs	Audit regressions
7	5 / 321	127 / 127	65,536 / 65,536	190 / 190	0
19	5 / 321	127 / 127	65,536 / 65,536	190 / 190	0
31	5 / 321	127 / 127	65,536 / 65,536	190 / 190	0

The acquired graph uses four XOR gates and one AND gate. Its emitted bit is the parity of the two input bits and old state. Its next state is their majority function, expressed through the acquired gate arrangement. A one-gate XOR foundation became this five-gate transition. On the eight-bit audit, correctness rose from 6,561 to 65,536 cases, with 58,975 gains and zero regressions per seed. The audit overlaps selection; the reserved bank and length-transfer bank remain independent.

The 321-byte figure is the serialized model, excluding interpreter, external teacher, search catalog and logs. Complete run wall time was 5.281 seconds, process CPU time was 5.219 seconds and peak process working set was 33.51 MiB. The shared function catalog took approximately 0.047 seconds to construct. Search, final execution and resource records are reported separately in the experiment record.

Seed	Foundation counterexamples	Repair counterexamples	Candidate simulations, both phases	Repair seconds
7	2	6	83,353	0.297
19	1	6	71,151	0.297
31	1	6	73,183	0.296

Counterexample counts omit successful verification exposures and cannot be described as the entire teaching set. Every accepted repair passed all 129 selection cases. The gate budget admitted 65,438 recurrent candidates; the external search evaluated candidates many times as new counterexamples arrived. Several candidates remained consistent with the accumulated counterexamples at acceptance, so the selected result also depends on the complexity ranking.

The final evaluator loaded the graph after a selection lock and never modified it. All eight local quantity-conservation identities passed a separate post-selection check. This supports the documented mathematical induction argument; the Python runtime itself has not been verified by a proof assistant. The trial demonstrates narrow procedure acquisition and structural repair under strong supplied assumptions, with no claim of superiority to a matched neural model.

4.6 Procedure acquisition and scaling

Two three-seed trials at revisions `a54d196` and `0453052` took 45.868 and 56.349 seconds on one CPU process. Each seed imported the earlier addition graph, learned subtraction from 48 examples, then acquired small programs. An independent audit checked all 723,668 final evaluation records across both trials, including base-only comparisons and failed cases. Peak worker memory was 48,963,584 and 50,249,728 bytes. Raw run files occupied about 20.1 and 20.7 MB at audit; those costs are separate from the compact model.

Initial seeds 7 and 19 retained 11 procedures in 2,004 bytes; seed 31 retained 12 in 2,131 bytes. Subtraction acquired five gates and passed 88 withheld cases, all 32,896 ordered eight-bit pairs and 32 length-transfer cases per seed. Every declared test passed for addition, subtraction, double, sum3, triple, adjusted difference, square, triangular sum and sum of squares. Multiplication passed its small domains but only 12 of 24 larger-input probes because its repeated-addition program loops over its first operand. Factorial failed from 8! onward under the iteration limit. Power was absent in two seeds and failed its larger probes in the third.

The follow-up inserted 24 large-quantity scaling examples under the unchanged search language and execution limits. Every seed acquired `scale(a, b) = multiply(b, a)` in nine candidates, adding 113 canonical bytes. New power and factorial programs called this acquired arrangement with a growing accumulator and a small count. All follow-up seeds passed 18 withheld, 117 audit and 16 transfer power cases; factorial passed 3 withheld, 11 audit and 4 transfer cases. The transfer ranges included 20 to the tenth power and 18 factorial. These are elementary calculations, not evidence of graduate reasoning.

The larger vocabulary also made sum-of-squares search exceed ten seconds in every follow-up seed. The base-only condition acquired it after 2,337 candidates. All such failures remain in the result. The follow-up changed the curriculum after observing the first trial; its new seeds and larger probes do not make it a blind replication. Earlier domain audits remain regression diagnostics. The complete protocol, partitions, per-seed measurements and source fingerprints are recorded in `experiments/2026-09-05-procedure-library.md` and its JSON companion.

4.7 Storage, reuse and retained procedures

Tripling needed one candidate with access to acquired sum3 versus 145 using only acquired addition and subtraction. Squaring needed one versus 49. Other tasks became slower with the larger vocabulary. Both conditions use the same task examples and budgets within a trial, but their three-instruction hypothesis classes differ in expressive power. The result is selective reuse, not a general advantage over alternative learners.

Later accepted follow-up programs added 109 to 171 bytes each. The 12-procedure shared library occupied 2,094 bytes versus 8,032 bytes for independently packaged dependency closures, a 73.9 percent reduction under that packaging control. The control is not an optimal compressed-code lower bound. Marginal storage did not decline monotonically or approach zero, and runtime cost could still grow rapidly.

A supplied packaging step appended the previously acquired sum-of-squares procedure to the follow-up library after verifying exact equality of its add, multiply and square dependencies. The retained artifact has 13 procedures in 2,238 bytes. Every existing follow-up definition remains unchanged. Eleven recorded queries and a further 1,089-case sum-of-squares audit passed after combination. This is preservation of compatible acquired work, not another learning trial or learned consolidation.

There were zero regressions across 4,506 protected case checks in the initial trial and 5,184 in the follow-up. Those totals count checks across stages, not unique inputs. Definitions were immutable; safe changes to shared dependencies remain untested. Original multiplication and its scaling wrapper still fail opposite halves of the large-operand-order probes, and the learner has no automatic dispatcher between them.

5 Closest mathematical relatives

5.1 Overall classification

The broader Kavi proposal is a dynamical system on computational graphs with feedback-dependent graph rewriting. Temporary signals and permanent structural changes have different roles. Self-Modifying Cartesian Genetic Programming is an operational relative because executable graphs include structural modification operations. Its learning extension investigated error-driven adaptation on small Boolean tasks; reliable transfer to all unseen truth tables was not achieved in that study.

The implemented instance is narrower: counterexample-guided synthesis of output and transition functions for a two-state Mealy transducer. Its learning controller and frame loop are engineered. This gives a concrete mathematical model for the first result without equating it to the whole adaptive architecture.

The current procedure extension is bounded typed program induction over acquired finite-state transducers. DreamCoder is a close library-learning relative: it searches programs, develops abstractions and learns to guide subsequent search. Kavi currently uses a fixed enumerator and supplied task signatures. It does not implement DreamCoder's learned search system or automatic abstraction invention.

5.2 Structural compression

The closest specific relative of Kavi's proposed compression is library learning modulo an equational theory. babble combines e-graphs and anti-unification to find reusable structure across programs whose syntax differs but whose expressions are equivalent under known equations. It starts with programs; it does not independently establish their correctness.

Stitch provides a related top-down search for abstractions that compress a program corpus. Its compression objective is relevant to a bounded learner, but shorter program descriptions do not automatically execute faster or improve unseen tasks. Those require separate measurements.

The E-Stitch workshop contribution, presented on 15 June 2026, directly combines top-down library learning with e-graphs. The official abstract describes preliminary results. It is a relevant recent direction; no design decision here depends on an unverified performance advantage from it.

5.3 Repair and verification

Syntax-guided synthesis makes the learning problem concrete through a grammar, a background theory and a correctness specification. Counterexample-guided inductive synthesis alternates between proposing a program and obtaining a case that refutes it. This fits Kavi's correction-driven development when the verifier can supply meaningful counterexamples.

An example-based verifier checks observed behavior. A solver may establish a universal property only within its supported logic and assumptions. Timeout, unsupported arithmetic or an incomplete search must remain distinct from a proof that no solution exists.

Stochastic superoptimization is a useful relative for searching small program mutations under behavioral and execution costs. Its original work concerns loop-free machine code and uses separate validation. Kavi would need its own instruction semantics and mutation operators. The published errata must be used when reproducing its cost equations.

5.4 Compression objective

Minimum description length gives a precise form to the preference for small reusable structure. The objective accounts for the model description and whatever data it fails to explain. The encoding must be specified: counting nodes while ignoring large constants, hidden lookup tables or a learned proposal network is insufficient. Grunwald's MDL introduction.

5.5 Quantum diagrams and physical dynamics

PyZX demonstrates automated rewriting of diagrams with quantum linear-map semantics. The transferable principle is that a structural rewrite should preserve a defined meaning. Kavi needs classical operational semantics for its programs. A quantum identity, particularly one valid only up to a global scalar, is not automatically valid for an exact classical numeric output.

The physical proposal has a different nearest relative: port-Hamiltonian systems on graphs, which combine energy storage, transport, dissipation and external inputs. That framework offers useful stability structure. It does not establish that simulated gravity, chemical fields or phase interference improve learning.

Kavi component	Closest mathematical family	Main distinction
Adaptive circuit objective	Feedback-dependent computational graph rewriting	Control and update-rule acquisition remain open
Implemented circuit	Counterexample-guided synthesis of a two-state transducer	Frame loop, encoding and state capacity are supplied
Acquired procedures	Typed program induction and library learning	Bounded calls and supplied iteration are implemented; general abstraction invention remains proposed
Structural consolidation	MDL and library learning modulo equations	Compression requires a defined encoding and valid identities
Behavioral repair	Counterexample-guided synthesis and program repair	Repair changes meaning and needs fresh correctness evidence
Equivalent-path optimization	Equality saturation	Optimizes represented alternatives, not all possible programs
Current text core	Complex nonlinear sparse recurrent network	Learns continuous coefficients and input/output maps
Fixed-size deployed circuit	Finite-state transducer under finite precision	Unbounded input families still require time and possibly workspace
Proposed physical core	Controlled dissipative graph dynamics	Stability and learning must be derived together

6 A formal specification for the intended learner

6.1 Operation learning and structural memory

The retained knowledge is the operation itself. An addition pathway combines quantities for new operands without retrieving remembered equations. Teaching examples may be kept by an external experimental harness, but they are not the deployed representation. Correction refutes the current procedure's contract and guides repair of the shared mechanism. A useful carry repair must improve unseen carry cases while preserving required earlier behavior.

For nonnegative unit quantities, let S introduce one additional unit. A recursive characterization of addition is:

$$A(a, 0) = a, \qquad A(a, S(b)) = S(A(a, b)).$$

These identities describe a reusable computation. Supplying that computation as an executable primitive would be a different learning task from discovering its arrangement. The current implementation supplies lower-level gates and a frame loop; future work should expose and measure the acquisition of control structure as well.

Let G be retained structure and s temporary execution state. The broader design permits input-conditioned temporary rewiring and feedback-dependent lasting updates. In the present implementation the graph is fixed throughout a query, and a successor graph is selected between teaching stages. Its concrete local transition returns an emitted bit e and new state c'. The independent post-selection check establishes:

$$e + 2c' = a + b + c \quad \text{for all } (a, b, c) \in \{0, 1\}^3.$$

Multiplying each frame's identity by its positional value and summing cancels adjacent state terms. With zero initial state and one final zero-input frame, the result is the sum of the two input integers. This argument assumes the documented executor and exact bit operations. The saved certificate checks the local identity exhaustively; it does not formally verify the implementation of the executor.

Wrong, provisional and verified are evidence statuses concerning an operation and its application. A failed example does not invalidate every subcomponent or imply that all earlier outputs were wrong. The live implementation handles exact generated feedback; learned interpretation of uncertain human feedback remains proposed.

6.2 Programs and types

Begin with a small typed language. Suitable initial types are Boolean, bounded integer, Unicode scalar, list of scalar, and tagged records. A Unicode scalar is a valid code point excluding the surrogate range; it is not necessarily a displayed character. Text tasks eventually need grapheme segmentation and normalization as explicit operations with declared versions.

Each primitive has a signature, deterministic semantics, a cost and defined failure behavior. For example, `head : NonEmptyList<T> -> T` cannot silently accept an empty list. Division must specify integer or rational arithmetic, rounding and division by zero. A path crossing from text into arithmetic must execute a parser whose possible failure is represented in its output type.

Use a directed acyclic graph for ordinary expressions and shared calls. Add iteration only through bounded loops or structurally decreasing recursion at first. A loop needs a termination argument or an execution limit. An edge joins compatible ports; it is not an unrestricted channel between arbitrary numerical vectors.

The active model is a library of acquired procedures together with task dispatch and executable program graphs. Denote it by M = (L, G). A separately learned proposal distribution q_phi may rank candidate edits. Its parameters and training costs are part of the learner's resource account whenever it is retained or required for learning.

6.3 Execution and state

Define execution explicitly:

$$\mathrm{Exec}_T(M, c, x) \in \{y, \mathrm{type\ \ error}, \mathrm{timeout}, \mathrm{runtime\ \ error}\}.$$

Here x is the input, c is declared context, and T is a budget. Context may identify a task through an instruction or a learned interpretation; it must not contain the correct answer. A finite set of task identifiers is acceptable for the first prototype, provided its use is reported. Generalizing from natural instructions is a later learning problem.

Distinguish persistent learned state from workspace. The library, graph, constants, routing statistics and proposal weights persist between lessons. Intermediate values, search queues and execution stacks are workspace. A replay set is external persistent memory. Historical checkpoints remain archival storage even when inference cannot read them.

Each current graph frame is a pure function with an explicit transition `(state, input) -> (state, output)`. The supplied executor sequences those transitions. Broader stateful operations should use the same explicit contract. Shared mutable state without such a contract makes equivalence, replay and concurrent path execution difficult to verify.

6.4 Optimization problem

For verified teaching and retention examples D, a practical deterministic objective is:

$$\min_{M} \; L_{\text{enc}}(M) + \lambda \sum_{i \in D} w_i C_{\text{exec}}(M, c_i, x_i)$$

subject to correct execution on D, a peak learning-memory limit, a total persistent-storage limit and a total search budget. The final evaluation set is excluded from D. Contradictory examples must be diagnosed rather than hidden by a nominally hard constraint.

L_enc is the bit length of a specified serialization. It includes primitive references, graph structure, constants, types, dispatch logic and learned proposal state. C_exec measures executed work, rather than the number of edges drawn in a diagram. The coefficient lambda expresses a declared engineering tradeoff; report results over a small predeclared range instead of selecting it on final tests.

For noisy tasks, replace exact agreement with a likelihood or a specified error tolerance. A coding prior P(M) proportional to 2 raised to minus L_enc(M) favors concise explanations. Combining it with an agreement likelihood produces a preference for compact consistent programs. This is a useful formal interpretation, not a claim that the optimal program can be computed efficiently.

6.5 Learning loop

1. Execute the current program and retain its actual trace.
2. Obtain a verified target or a counterexample. Record where that evidence came from.
3. Localize candidate edits using the trace, type constraints and earlier failure patterns.
4. Search a bounded neighborhood before expanding the grammar or capacity.
5. Reject ill-typed, nonterminating and behaviorally invalid candidates as early as possible.
6. Rank valid candidates by description length, execution cost and development-set behavior.
7. Test the selected change against retained contracts and new confirmation examples.
8. Install one explicit successor state and record the change. Preserve a recovery checkpoint outside the active model.
9. Search for reusable abstractions after enough distinct solutions exist. Verify each replacement before adopting it.

Trying candidates in temporary workspace is compatible with continuing one active learner. The deployed state can move forward through repairs without using old checkpoints as an inference ensemble. The search process still needs an explicit rule for what happens when no affordable candidate satisfies the constraints: retain the current behavior, return uncertainty, or request a more informative lesson.

6.6 A minimal example

Suppose the task context is either `first` or `last`, and the input is a nonempty list. A small grammar contains `head`, `reverse`, a context predicate and a conditional. Initial examples whose lists have one element do not distinguish the two tasks. A learner can fit them with `head(x)`.

The counterexample `last([0,1]) = 1` rules out that program. A consistent successor is `if last_context then head(reverse(x)) else head(x)`. This example shows why informative counterexamples matter: additional single-element examples would not expose the missing distinction.

This is an illustrative construction, not a result from the Kavi implementation. A finite exhaustive check over all nonempty binary lists of lengths one through five and both contexts covers 124 cases. Extending the claim to arbitrary finite lists requires the semantics of `head`, `reverse` and the conditional, not just that finite test. A later `last` abstraction may reduce description length while still requiring linear work in a singly linked representation.

6.7 Repair and consolidation have different obligations

A repair deliberately changes a function where the current function is wrong. Its obligation is to correct the target behavior and preserve the required surrounding behavior. A consolidation replaces an implementation with an equivalent one, or with one satisfying a declared distortion bound. A sound equivalence transformation also preserves existing mistakes; it cannot supply missing knowledge.

An e-graph stores expressions related by accepted equations. Equality saturation can make many alternative implementations available before selecting one using a cost model. It relies on the validity of its rewrite rules. Its compact representation can still consume substantial memory, so node limits, time limits and extraction cost belong in the experiment.

7 Capacity and the meaning of compression

7.1 A finite configuration contains finite information

A system with B bits of persistent state has at most 2^B distinct configurations. If it must memorize arbitrary labels from q possibilities for M independent inputs, there are q^M possible assignments. Representing every assignment exactly requires:

$$B \geq M \log_2 q.$$

This counting argument applies equally to weights, connection patterns, phases, coordinates, program text and constants. Calling a stored value a configuration does not remove its information content. A mathematical real number could encode infinitely many bits only under unbounded precision and exact operations; finite hardware does not provide them.

The constructive opportunity is structure. A short addition algorithm represents correct behavior on an unbounded family of finite integer inputs, while storing a separate answer for every pair would be impossible. Its code can remain fixed as inputs grow. Its runtime and working memory still depend on the representation and size of those inputs.

7.2 What should be compressed

Measure separate quantities: executable description length; parameter and optimizer bytes; retained examples; source and checkpoint storage; peak inference memory; peak learning memory; and learning and inference work. Reducing one quantity may increase another. Replacing a table with an expensive search saves persistent storage but may make the system unusably slow.

Shared subprocedures compress repeated structure. Quantization reduces coefficient precision and can alter outputs. Pruning removes representational choices. Distillation trains a successor to imitate a predecessor on sampled behavior. None of these establishes full knowledge preservation by itself.

The information bottleneck studies compressed representations that retain information relevant to a target variable. It is a useful conceptual relative for task-dependent compression. A target distribution and tradeoff are required; relevance to one task does not imply preservation of all future tasks.

7.3 Boundaries of finite state

With fixed finite-precision state and no growing external workspace, the deployed recurrent core is a finite-state machine, although its state space can be extremely large. Exact arbitrary-length copying or counting eventually exceeds that state capacity. Allowing a stack, tape, expandable list or retrievable archive changes the computational model and the memory budget.

The distinction determines which experiments are meaningful. Length extrapolation should report both correctness and workspace growth. A mechanism that reads an external answer table is retrieval. A mechanism

that executes an acquired procedure may generalize algorithmically. Both can be useful, but they answer different research questions.

8 Continual learning and preservation

8.1 Measure individual changes

After each update, partition evaluation cases into correct-to-correct, correct-to-wrong, wrong-to-correct and wrong-to-wrong transitions. Aggregate accuracy hides important differences: gaining ten answers and losing ten earlier answers yields no net change while failing a strict retention requirement.

Maintain three distinct reference sets. A training replay set can affect gradients. A development guard set can select or reject candidate changes. A final confirmation set is used only after the method and selection rule are fixed. Repeatedly checking a set and choosing updates based on its results makes it part of development.

For the intended program learner, exact preservation can sometimes be proved for a contract over an entire domain. For a neural learner, finite test preservation is generally the available evidence. Either way, state the covered input domain and any assumptions.

8.2 The implemented projection

The flow-preservation experiment computes an average reference-loss gradient g and projects the actual proposed parameter displacement d when its first-order effect is harmful:

$$d' = d - \frac{\max(0, g^{\mathsf{T}} d)}{\|g\|^2} g.$$

The implementation includes a near-zero denominator guard. This mechanism resembles A-GEM, which constrains a gradient using an episodic reference. Kavi applies its check to the optimizer's displacement; that detail matters when momentum and adaptive scaling are present.

The projection makes the first-order change in the average reference loss nonpositive. It does not guarantee that the finite update decreases that loss. If the loss has an L-Lipschitz gradient, the usual bound still contains a positive term `(L/2) * ||d'||^2`. Individual examples can also worsen while the average improves. Greedy decoded answers can change discontinuously when competing logits exchange order.

Use ordinary replay as a required control. Research on small episodic memories shows that simple replay can be competitive within studied continual-learning settings. Its effectiveness on Kavi must be measured with the same memory and update budgets.

8.3 Growth that preserves the initial function

Adding a residual connector with exactly zero initial amplitude can leave the initial network function unchanged. Duplicating a softmax route and subtracting log(2) from each duplicate's logit also preserves its combined contribution when source, phase, conductance and activity-dependent terms match. Perturbing or separately training those duplicates removes the guarantee.

These are function-preserving initializations, related in purpose to Net2Net. Their value is to create capacity without an immediate behavioral jump. They do not establish retention after subsequent learning. Record both the exact initialization check and the later behavioral results.

Dynamic sparse reparameterization reallocates connections during training under a parameter budget. Mostafa and Wang provide a concrete comparison for this direction. Connection count alone is not a speed measurement: sparse operations need suitable data structures and kernels to reduce executed work.

8.4 Context and shared structure

An update to a shared path can affect many tasks. Locality in the graph does not imply locality in behavior. Before editing a shared component, identify its callers and contracts. When two contexts require different outputs, a typed branch can separate them; when they share a rule, an abstraction can merge their implementation.

A collection of isolated task-specific modules may preserve old tasks by avoiding interference, but it can grow without bound. A single shared parameter pool may remain small while forgetting. The research target is a measured tradeoff among sharing, interference, capacity and search cost.

9 Physical dynamics and mathematical corrections

9.1 Separate the hypotheses

The existing complex recurrent core, the proposed physical dynamical core and the intended program learner are different mechanisms. A common vocabulary of paths does not make their learning rules interchangeable. Each proposed physical component needs an equation, a computational role, a numerical method and an experiment that can reject its usefulness.

Energy-based models describe an objective or dynamics through an energy function. Reservoir systems exploit recurrent dynamics with a restricted trained readout. State-space models parameterize temporal evolution. Program learners search executable structures. Kavi may combine these ideas later, but the simplest working versions should first be measured independently.

9.2 Gated mechanical dynamics

Consider the proposed deterministic equations with fixed mass, fixed input and no noise: q-dot = p/m and p-dot = -g grad(V) - gamma p. If the proposed energy is kinetic energy plus V, differentiation gives:

$$\dot{H} = \sum_i (1 - g_i) \frac{p_i}{m_i} \cdot \nabla_i V - \sum_i \gamma_i \frac{\|p_i\|^2}{m_i}.$$

The first term has no fixed sign. Gating the force alone can inject energy relative to that H. Positive damping does not remove this inconsistency.

A consistent starting point is a port-Hamiltonian form:

$$\dot{s} = (J_g - R)\nabla H(s) + Bu, \qquad J_g^{\mathsf{T}} = -J_g, \quad R \succeq 0.$$

With y = B-transpose grad(H), this gives H-dot = minus grad(H)-transpose R grad(H) plus y-transpose u. For position and momentum, a symmetric gate matrix G can appear in both off-diagonal blocks of `J_g = [[0,G],[-G,0]]`. The same gate then scales both coupled flows.

This is a direct design correction using the structure of port-Hamiltonian graph systems. It assumes the stated energy, fixed parameters and a continuous-time model. Changing masses, adding delays, injecting input, learning parameters or introducing reaction fields adds terms that must be accounted for. A numerical integrator needs its own stability analysis.

For a momentum equation with friction `-gamma p`, equilibrium Gaussian momentum with kinetic energy `p^2/(2m)` at temperature T requires noise amplitude `sqrt(2 gamma m T)` under the usual unit convention. Using `sqrt(2 gamma T)` instead produces momentum variance T, rather than mT. If temperature is merely an exploration coefficient, name and evaluate it as such; a thermodynamic interpretation is an additional claim.

9.3 Complex flow and numerical stability

For Hermitian H and positive semidefinite Hermitian Gamma, the continuous system psi-dot = (-Gamma - iH) psi is nonexpansive in squared norm. Its explicit Euler update is A = I - hGamma - ihH. Direct multiplication gives:

$$A^\dagger A = I - 2h\Gamma + h^2\left(\Gamma^2 + H^2 + i[\Gamma, H]\right).$$

Nonexpansion requires the last two terms together to be negative semidefinite. With Gamma = gamma I, a mode of frequency omega has squared amplification `1 - 2h gamma + h^2(gamma^2 + omega^2)`. For gamma greater than zero, the step must satisfy `h <= 2 gamma / (gamma^2 + omega^2)`. With zero damping, every nonzero frequency grows under explicit Euler for every positive step.

A Cayley step `U = (I + ihH/2)^(-1)(I - ihH/2)` is unitary for Hermitian H. Splitting damping around it, `exp(-hGamma/2) U exp(-hGamma/2)`, gives a contraction in the unforced case. Matrix solves and exponentials have computational costs; sparsity and locality are not automatic. Renormalizing an unstable Euler trajectory can conceal its numerical error while changing the dynamics.

An asymmetric type mask applied elementwise to H can destroy Hermiticity. Symmetrizing afterward can reintroduce a prohibited reverse connection. A clearer design uses reciprocal mixing inside compatible blocks and a separate operator for directional typed transport.

9.4 Equilibrium propagation

Equilibrium propagation relates a contrast between nearby free and weakly nudged equilibria to a parameter gradient. It assumes an appropriate differentiable energy and a regular stationary branch. The limiting statement does not establish accuracy for an arbitrary small number of settling steps.

For a total energy F(theta, s, beta), the parameter update uses the difference of partial derivatives with respect to theta at the two equilibria, divided by the nudging strength beta. Endpoint errors are consequently magnified when beta is small. Four free steps and two nudged steps require an empirical gradient-accuracy test; they cannot be declared sufficient by construction.

A correlation rule involving `q_i q_j` follows naturally from a bilinear interaction energy. A softened gravitational potential has a different parameter derivative. Derive the update from the actual energy used by the implementation. Parameters that only control how a solver reaches an equilibrium, such as a damping coefficient in an otherwise unchanged converged problem, need not affect the equilibrium objective at all.

First use a small positive-definite quadratic energy whose equilibrium and gradient are known analytically. Compare the proposed update with the analytic gradient, finite differences and automatic differentiation over a converged solver. Measure relative error and angle across nudging strengths, step sizes and settling tolerances.

A unique equilibrium depending only on the current input cannot retain an earlier sequence by itself. Temporal memory requires additional persistent state, history in the input, multiple stable states with controlled transitions, or a finite-horizon temporal objective. This choice must precede a language-learning claim.

9.5 Phase relations and quantum terminology

An outer product `rho_rs = psi_r conjugate(psi_s)` describes pairwise coherence for the chosen state representation. Physical entanglement requires a tensor-product subsystem definition and an appropriate nonseparability criterion. The original Werner paper supplies the relevant separability framework; only its abstract was inspected for this review.

A phase difference can encode a compatibility score or cause cancellation in a classical computation. It does not independently determine whether a statement is true. A contradictory hypothesis may cancel only if training or a specified rule has arranged that behavior. A claim of useful interference must compare learned phases with fixed, zero and randomized phase controls.

Quantum channels have additional mathematical constraints beyond a general complex update. Lindblad's generator framework is relevant only if that physical channel interpretation is intended; the inspected record was the article abstract. Ordinary classical graph computation does not need such a claim.

9.6 Recommended scope

Retain complex arithmetic as a testable representation choice. Put gravitational fields, chemical analogies, entanglement language and changing masses behind independent mathematical experiments. The first program-learning prototype needs none of them. A physical mechanism earns a place in the architecture when it improves a measured outcome after its extra state, integration work and tuning effort are counted.

10 Machine learning components and their roles

10.1 Input representation

Bytes provide a fixed vocabulary and exact encoding coverage, but one displayed character may require several bytes, and one grapheme may contain several Unicode scalars. Byte copying, scalar recognition, grapheme recognition, word segmentation and language comprehension are separate tasks. Kavi currently has byte and scalar mechanisms; neither establishes the later stages.

A learned tokenizer can shorten sequences but adds training data, vocabulary storage and segmentation behavior. A fixed byte model is a useful baseline because its preprocessing is transparent. For program induction, use structured values before adding a learned text parser. Evaluate the parser separately so a parsing error can be distinguished from an incorrect acquired procedure.

10.2 Representation and recurrent memory

The wave core embeds a byte, evolves a recurrent state and maps that state back to byte logits. Most parameters are in its input and output maps. Parameter-group ablations are therefore essential before attributing improvements to pathway topology.

Deep Complex Networks develops practical components for complex-valued learning and evaluates them in specific domains. Complex representations can be useful, but their real-valued block form means that complex notation alone provides no general expressivity or speed advantage over a suitably structured real system.

A reservoir baseline fixes recurrent dynamics and trains an output map, following the basic echo-state network approach. It tests whether recurrent dynamics need to be learned for these tasks. A small GRU and a linear recurrent unit provide additional controls. Linear Recurrent Units show how carefully parameterized recurrence can handle long sequences in their evaluated settings; Kavi still needs a matched local comparison.

10.3 Attention and routing

The original Transformer uses attention to combine information according to input-dependent scores. Kavi's four-slot soft routing is a much smaller local mixing mechanism. It has no comparable claim to global context access.

Soft routing computes all candidate contributions. Hard routing can save execution work by choosing a subset, but introduces discrete selection and load-balancing problems. Typed program dispatch offers exact executable control; learned routing can rank alternatives. Measure actual executed edges, routing errors and latency before claiming a sparse computation benefit.

10.4 Learning signals and optimization

Cross-entropy trains token probabilities. Exact-answer accuracy measures a different property. A lower average token loss can coexist with unchanged or worse exact answers. Report both, together with output length, invalid encoding and stop behavior.

Backpropagation is a method for computing derivatives, rather than an architecture or a synonym for memorization. Adam is an update rule using gradient moments. Alternatives must be compared against tuned, budget-matched gradient learning. Local learning may save activation storage while losing gradient fidelity or taking more steps.

Eligibility traces provide another way to organize temporal credit assignment. e-prop studies a factorization and approximation for recurrent spiking networks. Its rule cannot simply be copied into Kavi's complex recurrence without deriving the relevant state derivatives and learning signals.

10.5 Search and meta-learning

Program search supplies discrete structure that gradient descent does not directly choose. Enumeration, beam search, stochastic edits and solver-guided synthesis are reasonable first methods. All must include failed candidates and verification calls in their cost.

A learned proposal model can reduce search work after a corpus of verified solutions exists. It should rank candidates rather than certify them. Train it on development tasks and synthesized examples; compare it against uniform, frequency-based and type-directed ordering. Its training data and persistent size are part of the system.

Meta-learning becomes a meaningful description when earlier tasks improve the process of learning new tasks. Merely carrying a model across lessons is continual learning. Demonstrate meta-learning with disjoint task families and a measurable reduction in adaptation data or computation.

10.6 Reinforcement learning and planning

Reinforcement learning is appropriate when actions have delayed consequences and only reward is available. Exact arithmetic and copying already provide direct correctness signals, so reinforcement learning would initially add unnecessary variance. Later interactive tasks could use a program library as an action vocabulary and a planner to compose it.

For those tasks, specify environment state, observations, actions, reward, termination and exploration limits. Distinguish a learned world model from a hand-written simulator. Plan quality, learning cost and environment interactions require separate measurements.

10.7 Confidence and abstention

A softmax score is not automatically a calibrated probability of correctness. Guo and colleagues analyze calibration and simple post-training scaling in classification settings. For Kavi, evaluate calibration on held-out tasks and length ranges rather than importing their reported performance.

For program search, useful uncertainty signals include multiple consistent programs that disagree on a new input, failed verification, timeout and an unsupported type. An explicit abstention can be more informative than a confident fabricated result. It should count against coverage and be reported alongside conditional accuracy.

11 Software architecture

The gate learner and procedure extension implement bounded instances of the following contracts. A supplied teacher formalizes reviewed source material; the learner acquires circuits and small programs. General abstraction invention, learned proposal policies and consolidation across tasks remain extension points.

11.1 Component contracts

Component	Input	Output and invariant
Source register	Bibliographic record and local material	Versioned provenance and permitted teaching scope

Component	Input	Output and invariant
Lesson builder	Admitted source and prerequisite state	Verified examples with task and split identifiers
Input adapter	Raw text or structured input	Typed values or an explicit parsing failure
Program store	Accepted graph and library	Versioned, hashable model state
Executor	Typed program, context and values	Result, cost and actual execution trace
Candidate generator	Failure trace, grammar and budget	Well-typed candidate edits with provenance
Verifier	Candidate and specification	Counterexample, bounded test result, proof result or unknown
Learner	Verified feedback and candidate results	Proposed successor and resource accounting
Consolidator	Library and acquired programs	Equivalent or tolerance-bounded replacement
Evaluator	Frozen candidate and sealed tasks	Metrics without feedback into that candidate
Run controller	Reviewed configuration and user controls	Finite execution, checkpoint, pause and stop state

11.2 Model serialization

The current model uses a strict versioned JSON schema for ports, framing, gates, connections and output references. Unknown fields, invalid references and unsupported operations are rejected. Parent hashes, selection evidence, examples and search state remain external records. A future general model may extend the schema with type definitions, library entries and dispatch. Any retained learned proposal state must be counted. A source path alone is insufficient provenance; record content fingerprints.

Keep evaluation answers outside the model artifact. A training checkpoint may include optimizer and replay state, while a deployment artifact may omit them. Report both sizes. Loading a model must validate schema, dimensions, primitive references and resource limits before execution.

11.3 Teacher and learner separation

The teacher may read original material, generate examples and verify answers. The learner receives only the declared teaching signal. Teacher-written routes or target coefficients must be reported as supervision if supplied. Otherwise an apparent discovery can be an unreported transfer of the solution.

A learned text teacher is optional. Deterministic lesson generators with exact verifiers are preferable for the first algorithmic domains. Later source-based language lessons need reviewed extraction, ambiguity handling and independent answer checks. Teacher cost belongs in end-to-end learning cost when comparing teaching methods.

11.4 Module boundaries for implementation

The gate modules are `circuit_core`, `circuit_search`, `circuit_runtime` and `circuit_cli`. The procedure extension adds `procedure_core`, `procedure_search`, `library_curriculum`, `library_runtime`, `library_cli` and `learning_window`. The root CLI dispatches `python -m kavi circuit` and `python -m kavi library` to their respective interfaces. Richer type systems and discovered control semantics remain future work. Keep grammar, search policy and evaluation generators independently replaceable.

Avoid placing another large learner inside a CLI module. The existing `pathway_circuit` and `pathway_live` modules already concentrate substantial logic. New core behavior should expose small testable functions and explicit state transitions; command-line entry points should assemble configuration, invoke the core and present results.

12 Curriculum and source engineering

12.1 Prerequisites

Use a prerequisite graph rather than a fixed sequence of impressive subjects. A lesson is ready when the learner can perform the operations it depends on. Distinguish a failure to represent the input, interpret the instruction, execute the intended operation, or express the answer.

Stage	Teaching objective	Required transfer test
Signal fidelity	Preserve bytes, scalars and short ordered sequences	New symbols, positions and lengths
Primitive operations	Acquire selection, comparison and elementary transforms	New values within a declared domain
Composition	Combine acquired operations	New expression trees and deeper nesting
Reusable procedures	Extract patterns across solved tasks	Fewer examples or search steps on related new tasks
Symbolic arithmetic	Parse and execute arithmetic with exact semantics	New magnitudes, signs and expression structures
Controlled language	Interpret a small compositional grammar	New entity combinations and paraphrases
Source-based concepts	Learn reviewed concepts from original material	Independent applications and counterexamples
Broader language	Expand vocabulary, syntax and discourse	Separately designed comprehension and generation tests
Scientific reasoning	Model assumptions, derive consequences and test claims	New problems with independent verification

The 90% promotion policy is a project criterion, not an educational law. Report the denominator, task distribution, uncertainty and retention cost. A score of 90% on ten questions is nine answers; it provides much weaker evidence than the same rate across many independently designed task families.

12.2 Original sources

Maintain original language, author, work, edition, publication date, access location, license or access conditions, file hash, extraction method, reviewed pages and lesson scope. A source may be authoritative for its own historical argument while containing claims superseded by later science. Original publication status does not certify correctness or pedagogical suitability.

The local source collection inspected includes De Morgan's arithmetic, Baskervill and Sewell's English grammar, Unicode 17 data and a narrow algebra lesson extract. Admission of an extract is not admission of an entire book. The public manifests document scope while source bodies remain local.

The wider people-and-works catalog is a candidate collection. Selection should depend on conceptual value, verifiability, prerequisites, language accessibility and complementary perspectives. Reputation alone is an

unreliable syllabus. Original-language study should include competent language evaluation; translating everything into English and grading only English answers would test a different objective.

12.3 Lesson construction

Preserve the chain from source passage to concept, example, answer and verifier. Mark whether an answer was copied, derived by a deterministic program or reviewed manually. For historical or interpretive questions, allow a documented range of defensible answers instead of pretending an exact string is a complete verifier.

Generate contrastive cases that distinguish plausible rules. For sequence selection, vary the middle, length and endpoint independently. For arithmetic, test zero, sign, carries and boundaries. For language, separate changes in vocabulary, syntax and underlying relation.

Machine teaching formalizes the fact that teaching effectiveness depends on the learner and hypothesis space. Teaching dimension for linear learners is a precise example of this dependence. It does not establish that a short human explanation will efficiently teach an arbitrary recurrent network.

12.4 Split integrity

Split tasks by generative structure where possible, rather than only by random examples. A new number in a familiar template tests value generalization. A new composition tree tests structural generalization. A new language, source work or instruction style tests a different shift.

Record exposure whenever a reserved question is shown during correction. Keep its historical result, but remove its claim to being unseen. A fresh final bank should be created from a predeclared generator or independently authored specification after selecting the method, then sealed for confirmation.

13 Experiments that can resolve the main questions

13.1 Baseline reproduction

Freeze a code revision, environment record, model checkpoint, teaching order and evaluation generator. Reproduce one existing text comparison before modifying architecture. Store per-example outputs and correctness transitions, not just rounded percentages.

Use the current 66,880-parameter wave model, a parameter-matched small real recurrent model, a GRU, a reservoir with trained readout and a small linear recurrent unit. Matching parameter counts and matching runtime answer different questions; report both comparisons. A small attention baseline becomes useful once sequence lengths justify its setup cost.

13.2 Program-learning experiment

The first Boolean transition experiment is implemented and measured in section 4.5. The next acquisition experiment should use a grammar that can express several small list, string and integer transformations without containing those target solutions as individual primitives. Start with exact pure functions and a strict execution budget. Hold out both examples and composition structures.

Compare type-directed enumeration, stochastic local edits, enumeration with a learned proposal order, and the same methods with learned library abstractions. Include a supplied-program upper control to quantify execution and evaluation overhead. Label it clearly so it cannot be mistaken for learning.

The decisive outcome is a new task solved with fewer examples or less search because an earlier acquired abstraction is reused. Record the abstraction, the actual call trace and the baseline's cost. A shorter stored program without transfer is evidence for compression only.

13.3 Experimental matrix

Question	Intervention and control	Main measurement	Failure criterion
Do learned pathways matter	Train all groups versus input/output maps only and routing only	Exact accuracy and parameter-group change	No repeatable benefit from learned routing
Does complex phase help	Learned phase versus zero, fixed random and real block controls	Accuracy per time and memory	Advantage disappears under matched tuning
Does topology help	Fixed graph versus rewiring and exact-initialized splitting	Transfer and executed work	More state or time without reliable gain
Does retention projection help	Continuation, replay and displacement projection	Old breaks and new corrections	Worse tradeoff than ordinary replay
Do repair connectors help	No connector versus 8 and 9 connectors	Paired per-case changes	Differences limited to wrong output strings
Does library learning help	Same search with and without abstractions	New-task search cost and serialized size	Compression without useful transfer
Are rewrites sound	Exact or solver-checked equivalence versus invalid controls	Counterexamples and proof coverage	Any accepted invalid rewrite
Are physical dynamics stable	Analytic small system versus proposed integration	Energy residual and step-size sensitivity	Unexplained energy growth or solver dependence
Is local learning accurate	Proposed update versus analytic and gradient references	Relative gradient error and learning cost	Error remains large as settling improves
Is curriculum selection useful	Fixed, random, interleaved and error-targeted teaching	Area under learning curve and retention	Gains rely on exposed test answers

13.4 Retention protocol

For every method, initialize from the same checkpoint for a paired seed. Keep the teaching budget, replay bytes and evaluation schedule equal. Report the union of historically correct cases, the currently correct cases and independently sampled earlier-domain cases separately.

A zero-failure guard result is useful but limited. Under independent identical Bernoulli sampling and no adaptive selection, zero failures in n trials gives a one-sided 95% upper bound `1 - 0.05^(1/n)` on failure probability. For n = 196 this is approximately 1.52%. The actual 196-case consolidation guard was used for selection, so that simple bound is not a valid generalization guarantee for the selected candidate.

For a claim of no forgetting over a finite supported domain, exhaustively enumerate the domain. For an unbounded domain, seek a proof under specified semantics or narrow the claim. A sampled regression suite cannot establish a universal theorem.

13.5 Seeds and statistical reporting

The existing three-seed comparisons are exploratory, especially because their continuations share a parent checkpoint. They characterize some update variability, not the full variability of learning from independent initialization.

For a confirmatory study, a practical starting design is ten paired seeds with independent model initializations, subject to a pilot-based compute budget. Predeclare the primary metric, minimum useful effect and comparison. Use paired differences and intervals; do not treat hundreds of questions from one trained model as hundreds of independent model replications.

Report per-task results, medians or means as appropriate, and uncertainty across both tasks and seeds. Hierarchical resampling can preserve that structure. The general lesson of Agarwal and colleagues' evaluation study is relevant: small benchmark collections and few runs can produce unstable conclusions. Its reinforcement-learning results are methodological guidance rather than Kavi measurements.

Account for all attempted candidates and tuning runs. A winning run found after extensive search is not comparable to a baseline given one setting. Publish failures and stopping rules alongside the selected configuration.

13.6 Metrics

Report exact-answer accuracy, token loss, per-task accuracy, length extrapolation, invalid outputs, abstention coverage, calibration where meaningful, learning curves, examples to threshold, correct-to-wrong transitions and newly corrected cases. For program learners, also report solved tasks, verified domain, search expansions, verifier calls, abstraction reuse and description length.

For resources, record wall time, process CPU time, peak resident memory, parameter bytes, optimizer bytes, replay bytes, checkpoint bytes and total persistent storage. Distinguish process from whole-device measurements. If temperature cannot be read reliably, record it as unavailable.

Benchmark suites can provide a later independent test. The ARC-AGI-2 specification and the ARC Prize 2025 technical report are useful references for novel-task evaluation and controlled competition evidence. A score on an abstraction benchmark is not a direct measure of language fluency or broad real-world competence.

The Tiny Recursive Models study supports investigating repeated computation in small task-trained models. Its results do not imply that Kavi's much smaller current network can acquire the same capabilities. Architecture, supervision, search, compute and benchmark conditions all differ.

14 Engineering and operation

14.1 Current environment

The inspected device has an Intel Core i7-10870H with eight physical cores and sixteen logical processors, approximately 31.78 GiB of reported system memory, and a GeForce GTX 1650 Ti with approximately 4 GiB of graphics memory. The verified project interpreter is Python 3.13.5 with PyTorch 2.6.0+cu124. The current experiments use CPU execution; installed CUDA support does not mean a run used the GPU.

All 169 automated tests passed after the procedure and scaling extensions. The suite covers circuit acquisition, executor consistency, isolated serialization, invalid graphs and libraries, nested iteration, partial resource accounting, large-quantity call-order acquisition, preserved definitions, source fingerprints and run controls. The first gate run used Python 3.12.14; both library runs used Python 3.13.5. The earlier 138- and 153-test results remain historical evidence.

The repository now resides in `C:\Users\admin\Desktop\PI&E`. The move preserved 1,357 files and 228,925,215 bytes, with per-file SHA-256 verification. This total includes local run state and private material as well as source code. Private source and run folders remain excluded from public version control.

14.2 Development commands

Run commands from the repository root with the intended Python environment. The package declares Python 3.11 or later and an optional `wave` dependency for PyTorch. The fully exercised environment is the version listed above; the declared version range is broader than the tested configuration.

```
python -B -m unittest discover -s tests -q
python -m kavi --help
python -m kavi circuit --help
python -m kavi library --help
python -m kavi.pathway_cli --help
python -m kavi.wave_cli --help
```

The structural entry point is `python -u -m kavi circuit run --config curriculum/circuit-run.json --run-dir runs/circuit-trial --interactive`. It requires a new directory, runs under finite limits, and then opens a query console without continuing automatic teaching. The launcher `scripts/start-circuit.ps1` selects a timestamped directory. Commands `watch`, `status`, `control`, `ask`, `inspect` and `console` expose the run and saved graph. Actual gate traces are available with `--trace` or `/trace`.

The live launch scripts create teaching processes and terminal views. They are operational entry points, not installation checks. Inspect their arguments and the curriculum before beginning a run. A read-only watch process and a training process have different effects.

14.3 Resource control

Keep explicit limits for steps, wall time, CPU threads, candidate count, checkpoint retention and disk use. A model with less than one megabyte of parameters can still require hundreds of megabytes of process memory because the runtime, optimizer, tensors and experiment controller also occupy memory.

On this device, the recorded comparison peaks near 0.5 GiB show that these small trials fit comfortably within process memory. They do not establish that future search, source ingestion or a population of models will fit. The number of concurrently resident candidates must be included in estimates.

Use reliable measured temperature only when a sensor is available. A historical shutdown temperature is not an operating target. Preserve pause and stop controls, atomic checkpoint writes and bounded recovery attempts. The structural trial ran under its declared finite CPU configuration. Earlier text training was not restarted and hardware policy was not changed.

14.4 Reproducibility record

Each experiment should retain code revision and diff, interpreter and dependency versions, device and thread settings, task-generator revision, all random seeds, starting model hash, source fingerprints, configuration, candidate-selection rule, budgets, stopping reason, per-case outputs and resource measurements.

Absolute paths are machine-specific. New configurations should resolve project resources relative to the repository or accept explicit paths. Historical records may retain their original paths as evidence of the run environment; changing those strings would not replay the experiment.

Use a small, versioned public result summary with enough detail to assess the claim. Keep raw source material, conversations, checkpoints and full runtime logs in their designated local folders. Software licensing remains an unresolved repository decision; third-party citations and source conditions retain their own attribution.

15 Development milestones

15.1 Sequence and acceptance criteria

Milestone	Deliverable	Acceptance criterion	Planning allowance
M0 Structural baseline	Frozen circuit experiment, resource record and independent final banks	Recorded three-seed run, portable model and per-case evidence	Completed 5 September 2026

Milestone	Deliverable	Acceptance criterion	Planning allowance
M1 Typed program core	Interpreter, grammar, bounded search and exact verifier	Acquire small arithmetic programs from examples without supplied target arrangements	Bounded arithmetic instance measured 5 September 2026
M2 Compositional transfer	Shared library and new task families	Lower adaptation cost under disclosed comparisons and preserved behavior	Selective arithmetic gains measured; broader test remains open
M3 Continual repair	Local edits, retention contracts and failure handling	Correct new tasks with a measured improvement in retention-cost tradeoff	3 to 6 weeks
M4 Verified consolidation	Sound rewrites and explicit description-length accounting	Smaller complete artifact with preserved covered behavior	2 to 4 weeks
M5 Controlled language interface	Parser, ambiguity handling and source-linked lessons	New compositions and independently reviewed language tests	Several months
M6 Broader research validation	External task suite and independently reproducible results	Advantage survives task, seed and budget controls	Depends on earlier evidence

The gate experiment supplies a finite-domain shared repair result relevant to M3. The procedure trials add a bounded M1 implementation and mixed evidence for M2: useful reuse alongside search slowdowns and budget failures. Broader types, discovered abstractions and new algorithm families remain untested. Remaining time ranges are planning allowances, not predictions of when a capability will appear; they exclude failed research hypotheses.

15.2 Priority decisions

Extend the measured program learner toward efficient algorithms, better proposal ranking, richer types and controlled acquisition of branches. Preserve earlier implementations and original failed trials as comparison evidence. Separate changes to shared behavior from equivalent compression and append-only growth. Count any numerical proposal model if one is added. Establish execution and search costs before adding elaborate physical state.

Do not expand the curriculum merely because a current gate is difficult. Diagnose whether the limitation is input representation, credit assignment, capacity, search, teaching examples or evaluation design. More source text does not resolve an unidentified architectural failure.

15.3 Stop and redirect rules

If library learning improves compression but not transfer, retain it as a storage result and revise the task distribution or abstraction search. If a physical mechanism matches a simpler model only after more tuning and compute, exclude it from the primary system. If retention requires linear growth in isolated modules, report the tradeoff rather than calling it bounded consolidation.

A negative result with a precise mechanism and controlled experiment is useful research. The project should stop expanding a mechanism once its declared tests have repeatedly failed and there is no new explanatory hypothesis.

16 Prospects and attainable levels

16.1 Present level

Kavi now has a measured structural operation learner and bounded procedure acquisition. A retained artifact combines 13 acquired operations in 2,238 bytes. It executes learned circuits and programs without retrieving teaching equations. A small acquired scaling arrangement enabled useful downstream power and factorial programs, while other search and execution failures remain documented.

This is a narrow implemented instance of the intended architecture. Representation, state capacity, types, instruction semantics and the learning controller are supplied. Acquiring those semantics, inventing abstractions across substantially different tasks and interpreting language remain open. Earlier symbolic execution and limited text learning remain separate evidence. No university or master's-level subject competence has been established.

16.2 Next research milestone

The next research milestone is to improve algorithmic efficiency and abstraction selection while preserving a continuing library. Repeated addition is too expensive for multiplication of two large operands. Library growth can make naive enumeration slower. A useful extension must address these measured bottlenecks, then demonstrate transfer to new task families and safe changes to shared dependencies.

The potential contribution lies in a specific combination: typed local repair, reusable structure, measured consolidation and explicit resource limits in one continuing learner. Those individual ideas have extensive prior art. Novelty would need to reside in a new mechanism, a formal result or a convincing empirical tradeoff, rather than in their names or combination alone.

16.3 Practical specialist

If those results hold, the next plausible level is a specialist for verifiable transformations: structured text manipulation, small data-cleaning procedures, algebraic transformations within a supported theory, or adaptation of a finite workflow from examples. Such a system can be useful through reliability, inspectability and low adaptation cost, even with a narrow task vocabulary.

Reaching that level requires robust parsing, explicit unsupported cases, predictable latency, persistence, regression testing and a clear interface. A compact research model that occasionally succeeds on generated examples is not yet a dependable tool.

16.4 Broader capability

Language, scientific reasoning and open-ended problem solving add ambiguity, world knowledge, long context, noisy supervision and difficult verification. Typed program structure may help organize acquired skills, but it does not supply those capabilities automatically. A hybrid with learned perception and language interpretation is plausible; its extra models and data must be counted.

A laptop can support useful experiments and a narrow deployed learner. Current evidence does not justify forecasting frontier-level general intelligence, universal no-forgetting, unlimited memory or the solution of major open mathematical problems. No defensible numeric probability of those outcomes follows from the repository.

An advanced mathematics and computer science target requires proofs, unfamiliar algorithmic problems, statistical reasoning, model selection and independent research work. Official graduate course descriptions in advanced algorithms and machine learning provide scope references. The project protocol in `docs/ADVANCED_CAPABILITY_PROTOCOL.md` separates teaching exposure, independent task families, exact checks and expert assessment. No automatic arithmetic score confers that capability label.

16.5 Conditions for success

The project has a realistic path to a useful research contribution if it narrows the first claim, implements the actual learned object, publishes reproducible comparisons and treats failed tests as design information. Its prospects weaken when additional analogies are added faster than existing mechanisms are verified.

The best near-term target is a result that can be stated in one measurable sentence: under the same total budget, learned typed abstractions reduce the cost of acquiring new procedures while preserving a specified set of earlier behaviors. Demonstrating that result would establish a sound basis for deciding how far to scale.

17 Technical study sequence

Study the mathematics in the order required by implementation. Linear algebra and numerical differentiation support the current recurrence. Probability and experimental design support valid comparisons. Type systems and operational semantics support the intended executable representation. Search, constraint solving and MDL support acquisition and consolidation. Dynamical systems and numerical integration support the optional physical core.

Topic	Working understanding required	Concrete exercise
Linear algebra	Complex real-block maps, eigenvalues, singular values and conditioning	Verify the wave recurrence against an equivalent real implementation
Numerical optimization	Gradients, Adam state, clipping and finite updates	Compare analytic, finite-difference and automatic gradients
Sequence learning	State, truncation, teacher forcing and length transfer	Measure copying as length and recurrent state size vary
Probability	Likelihood, priors, calibration and conditional evidence	Compare several programs consistent with the same examples
Experimental design	Paired seeds, task splits, tuning cost and uncertainty	Reproduce one trial with a sealed final evaluation
Type systems	Signatures, composition, effects and error values	Build a typed interpreter with explicit failures
Program synthesis	Grammar search, counterexamples and solver limits	Acquire list transformations without exposing target programs
Compression	Coding length, shared abstractions and execution tradeoffs	Replace repeated subprograms and measure complete artifact size
Continual learning	Interference, replay, constraints and domain coverage	Record all four correctness transitions after each lesson
Dynamical systems	Stability, energy balance and numerical integration	Compare Euler and a stable reference on a known quadratic system
Formal verification	Finite tests, invariants, equivalence and proof assumptions	Prove or exhaustively verify a small rewrite rule
Systems engineering	Serialization, profiling, reproducibility and process control	Recover a finite run from a validated checkpoint

Each exercise should produce a small executable result and a failure case. Reading alone will not expose whether an equation, verifier or abstraction behaves as intended in the actual system.

Appendix A Implemented component inventory

The current package contains 62 modules. Four implement the discrete gate learner, six implement procedure acquisition and its interface, and 52 describe the earlier systems.

Module	Responsibility
`circuit_core.py`	Strict graph format, streaming bit execution and actual traces
`circuit_search.py`	Expression catalog, graph compilation and counterexample-guided search
`circuit_runtime.py`	Teacher, run controls, protected behavior and independent evaluation
`circuit_cli.py`	Live run, queries, inspection, watching and controls
`procedure_core.py`	Strict library format, acquired calls, iteration, serialization and traces
`procedure_search.py`	Bounded expression search, temporary caches and all candidate work
`library_curriculum.py`	Source witnesses, formal lessons and final evaluation generators
`library_runtime.py`	Acquisition, vocabulary comparisons, storage, controls and sealed results
`library_cli.py`	Run, inspect, query, console, status and controls
`learning_window.py`	Visible worker, actual transcript, pause/stop and saved queries

The following inventory covers all 52 Python modules under `kavi` at the inspected baseline. Earlier cores are retained as separate experiments; they should not be added together as though they constituted one trained model.

A.1 Initial pathway and explanation experiments

Module	Responsibility
`types.py`	Event, pathway, trace and update data contracts
`graph.py`	Initial routed graph, path selection and numeric execution
`learning.py`	Verification and candidate pathway updates for generated arithmetic
`runtime.py`	Finite event loop, controls, measurements and persistence
`cli.py`	Root command dispatch and earlier stage-0 interface
`lessons.py`	Structured teaching explanations and lesson examples
`explanation_learning.py`	Translation of supplied explanations into checked updates
`lesson_runtime.py`	Finite explanation-learning experiment loop
`lesson_cli.py`	Explanation experiment commands
`__init__.py`	Package metadata and import boundary
`__main__.py`	Entry point for `python -m kavi`

A.2 Symbolic learning and composition

Module	Responsibility
`symbol_core.py`	Trainable signal prototypes for early symbol tasks
`symbol_runtime.py`	Finite symbol curriculum and recorded evaluations
`unicode_core.py`	Unicode scalar representation and small script-route model
`unicode_runtime.py`	Generated scalar and script curriculum controls
`textbook_core.py`	Compact numeric concept core for the reviewed algebra lesson
`textbook_runtime.py`	Source fingerprint checks and finite concept teaching
`school.py`	Prerequisite and promotion orchestration across early cores
`school_cli.py`	Curriculum inspection and finite school commands
`adaptive_syllabus.py`	Adaptive checks, diagnosis and repair queues
`adaptive_cli.py`	Adaptive syllabus command-line interface
`pathway_circuit.py`	Unified routes, prototypes, adapters and typed composition execution
`pathway_live.py`	Cross-stage teaching, checkpointing and multiple event feeds
`pathway_cli.py`	Unified circuit commands and feed access
`composition_curriculum.py`	Supplied structural contracts and composition teaching cases
`composition_evaluation.py`	Separate composition audit generator
`developmental.py`	Supported mastery checks, correction rounds and fresh tests
`teaching_search.py`	External candidate-update comparison and selection
`script_reference.py`	Teacher access to fingerprinted Unicode script data

A.3 Text learning and sources

Module	Responsibility
`wave_core.py`	Complex recurrent byte model, loss, optimization and generation
`continuous_teacher.py`	Resumable book teaching and interaction queues
`language_teacher.py`	Language prerequisites and answer-focused corrections
`language_curriculum.py`	Small generated language tasks and exact targets
`book_curriculum.py`	Source admission, arithmetic units and exam partitions
`mixed_quizzes.py`	Fresh copying, joining and sequence-position tasks
`multilingual_bridge.py`	Small writing-system subsets, mixed quizzes and retention

Module	Responsibility
`wave_cli.py`	Text run controller, read-only feeds and local console
`source_manifest.py`	Source records, fingerprints and teaching-scope validation
`source_cli.py`	Source admission inspection
`teaching_sources.py`	Reviewed-source lookup and bounded teaching packets
`catalog_cli.py`	Read-only people-and-works catalog interface

A.4 Experiments and common infrastructure

Module	Responsibility
`strategy_trials.py`	Teaching recipes and comparison partitions
`teaching_comparison.py`	Isolated comparisons of teacher methods
`pathway_trials.py`	Damped updates, rewiring and route-splitting candidates
`repair_trials.py`	Small context-dependent residual connectors
`flow_preservation.py`	Reference-gradient displacement projection
`consolidation_trials.py`	Bounded interpolation search with guard verification
`forward_repair.py`	Continued learning from the latest configuration and jump variants
`trial_resources.py`	Process resource observations and available sensor readings
`file_io.py`	Atomic writes with Windows reader-lock handling
`terminal.py`	Terminal encoding and output compatibility
`friendly_live.py`	Readable presentation of recorded lessons and computations

The `scripts` directory contains launchers, checkpoint comparisons and finite experiment drivers. `curriculum` contains source manifests, teaching policies and generated-task definitions. `tests` contains implementation regressions. `experiments` contains public measurement records. `runs` and `private` hold local evidence and material excluded from public version control.

Appendix B Additional design hypotheses

Earlier design notes contain mechanisms that are not part of the current implementation. They can be retained as a research backlog with concrete interpretations and rejection tests.

Proposed mechanism	Computational interpretation	Required test before integration
Knowledge mass	Importance-dependent resistance to parameter or structural change	Retention benefit beyond replay, with adaptation cost measured
Attraction between concepts	Explicit similarity or binding potential	Better associations than a simple similarity kernel

Proposed mechanism	Computational interpretation	Required test before integration
Momentum and orbit-like motion	Recurrent persistence or optimizer momentum	Useful temporal behavior under stable integration
Temperature	Exploration scale or stochastic perturbation	Improvement over a fixed or scheduled noise baseline
Local clocks	Different update frequencies for components	Equal-quality reduction in executed work
Reaction and inhibition	Competing activation and suppression fields	Stable selective activity with fewer operations
Propagation delays	Bounded queues or delayed state connections	Better temporal tasks after delay-memory cost is counted
Structural fission	Function-preserving capacity allocation followed by specialization	New learning with measured retention and storage cost
Structural fusion	Verified replacement of redundant computation	Preserved behavior and lower complete description length
Fast and slow memory	Short-lived workspace plus persistent consolidated state	Transfer and retention beyond equal-budget single-store controls
Reversible branches	Temporary candidate states with controlled commit	Lower search cost than ordinary copying or persistent data structures
Homeostasis	Constraints on state scale and activity	Stability without suppressing useful distinctions
Multiscale event formation	Learned grouping of input units	Better sequence cost and generalization than fixed preprocessing
Sparse experts	Conditional execution of specialized operators	Useful specialization with bounded routing overhead
Retrieval	Search over explicit stored facts or examples	Correctness and latency with storage counted separately
Population search	Bounded candidate diversity and selection	Gains after all candidates and tuning costs are included
Information-gain teaching	Choose lessons that separate plausible hypotheses	Fewer lessons than random or error-only selection
Code revision search	Propose and verify changes to implementation	Isolated later experiment with fixed evaluator and operating limits
Formal mathematics	Typed statements, derivations and proof checking	Valid proofs in an explicitly supported formal system

An execution trace is evidence of what ran. A natural-language explanation is an interpretation of that trace. For mathematical work, a proof accepted by a trusted checker establishes the encoded theorem under the checker's assumptions; it does not verify that the encoded statement matches an informal claim without further review.

Appendix C Evidence boundaries

Claim	Evidence status on 5 September 2026
Internal text parameters change from teaching	Implemented and observed in recorded experiments
Earlier text core has 66,880 base parameters	Derived from inspected shapes and checked model accounting
Regression suite	138 after relocation; 153 after gate learning; 169 after procedure and scaling extensions
Longer copying generalizes reliably	Not supported by the recorded five-symbol results
Small repair connectors eliminate forgetting	Refuted by the recorded correct-to-wrong transitions
All 196 selected guard answers can be retained	Observed for one selected consolidation fraction
That guard proves universal retention	Unsupported; two old correct answers broke on final confirmation
An extra forward jump improves final correctness	Not observed in the paired three-seed comparison
Typed composition executes supplied contracts	Implemented and checked within a bounded curriculum
Gate-level operation acquisition and repair	Implemented and measured under a supplied streaming executor
Bounded program acquisition and acquired calls	Implemented and measured; supplied types and iteration semantics
General control semantics and abstraction invention	Unimplemented
Compact retained procedure library	13 acquired procedures, 2,238 bytes; supplied combination after exact dependency checks
Library growth always improves acquisition	Contradicted by measured search slowdowns and timeouts
General library compression preserves learned knowledge	Proposed; requires specified semantics and experiments
Physics-inspired dynamics outperform simpler models	Unmeasured
Finite resources can hold arbitrary unlimited information	Incompatible with finite-state counting
A compact procedure can cover an unbounded input family	Mathematically possible with appropriate execution resources
Broad language, general intelligence or research autonomy	Not demonstrated

Appendix D Research references

Sources were inspected on 5 September 2026. The scope column distinguishes full-paper access from abstracts and official summaries. Direct mathematical derivations in this specification concern the stated Kavi equations and assumptions; they are not attributed as experimental findings of the cited papers.

Reference	Relevance and inspected scope
Harding, Miller and Banzhaf 2007 Self-Modifying Cartesian Genetic Programming	Executable graphs with structural modification operations; primary paper
Harding, Miller and Banzhaf 2009 Evolution, Development and Learning Using Self-Modifying Cartesian Genetic Programming	Error-driven graph adaptation, finite Boolean tasks and incomplete unseen-task transfer; primary methods and results
Ellis et al 2021 DreamCoder	Program synthesis with learned libraries and search; full paper
Lake et al 2015 Human-level concept learning through probabilistic program induction	Structured priors for concept acquisition; full paper
Grunwald 2004 A Tutorial Introduction to the Minimum Description Length Principle	Description-length model selection; full tutorial
Alur et al 2013 Syntax-Guided Synthesis	Grammar, specification and counterexample-guided search; full paper
Willsey et al 2021 egg	Equality saturation and e-graph engineering; full paper
Bowers et al 2023 Top-Down Synthesis for Library Learning	Stitch abstraction search; full paper
Cao et al 2023 babble	Equational library learning and anti-unification; full paper
Gupta et al 2026 E-Stitch	Recent combination of top-down search and e-graphs; official workshop abstract only
Schkufza et al 2013 Stochastic Superoptimization	Search over executable mutations; full paper and errata
Kissinger and van de Wetering 2020 PyZX	Semantics-based diagram rewriting; full paper, QPL 2019 work
van der Schaft and Maschke Port-Hamiltonian Systems on Graphs	Energy balance in graph dynamics; preprint and published-work record
Scellier and Bengio 2017 Equilibrium Propagation	Equilibrium gradient relation and assumptions; full paper
Jaeger 2001 The Echo State Approach	Fixed recurrent reservoir and trained readout; institutional report record
Trabelsi et al 2018 Deep Complex Networks	Complex neural components; full paper, 2017 preprint
Orvieto et al 2023 Resurrecting Recurrent Neural Networks for Long Sequences	Linear recurrent unit baseline; proceedings and paper
Chaudhry et al 2019 Efficient Lifelong Learning with A-GEM	Average episodic gradient constraint; full paper

Reference	Relevance and inspected scope
Chaudhry et al 2019 On Tiny Episodic Memories in Continual Learning	Replay control; primary abstract and study record
Mostafa and Wang 2019 Parameter Efficient Training of Deep Convolutional Neural Networks by Dynamic Sparse Reparameterization	Fixed-budget structural changes; proceedings and paper
Chen et al 2016 Net2Net	Function-preserving initialization; primary abstract and paper record
Bellec et al 2019 A Solution to the Learning Dilemma for Recurrent Networks of Spiking Neurons	Eligibility traces and learning signals; full preprint
Werner 1989 Quantum States with Einstein-Podolsky-Rosen Correlations	Separability distinction; abstract only
Lindblad 1976 On the Generators of Quantum Dynamical Semigroups	Physical channel constraints; abstract only
Tishby et al The Information Bottleneck Method	Task-relevant compression; primary abstract, 1999 work uploaded in 2000
Vaswani et al 2017 Attention Is All You Need	Attention baseline; primary paper record
Guo et al 2017 On Calibration of Modern Neural Networks	Confidence calibration; proceedings and paper record
Liu and Zhu 2016 The Teaching Dimension of Linear Learners	Learner-dependent teaching requirements; journal record
Agarwal et al 2021 Deep Reinforcement Learning at the Edge of the Statistical Precipice	Few-run evaluation reliability; proceedings record
ARC-AGI-2 2025	Independent abstraction-task design; primary paper record, revised 2026
ARC Prize 2025 Technical Report	Competition evaluation and refinement evidence; January 2026 report record
Jolicoeur-Martineau 2025 Less Is More	Tiny recursive model research; primary paper record

Appendix E Repository evidence

The earlier code analysis uses commit `40a54f9b25fe7a8685e59f635d99f3fa3805a23d`. Gate acquisition uses `7908156e0241a9df4293e56231660fb234ca44de`; the library trials use `a54d196` and `0453052`. The public circuit and procedure-study records include compact JSON evidence and saved acquired models. Source fingerprints and full per-case records remain with the local runs. The evidence includes:

- `experiments/2026-09-05-procedure-library.md`
- `experiments/2026-09-05-circuit-learning.md`
- `experiments/2026-09-04-teaching-and-pathways.md`
- `experiments/2026-09-04-small-repair-connections.md`
- `experiments/2026-09-04-verified-consolidation.md`
- `experiments/2026-09-04-forward-repair.md`
- `experiments/2026-09-04-developmental-teaching-live-run.md`
- `experiments/2026-09-04-language-first-repair.md`
- `experiments/2026-09-04-contrast-teaching.md`

Private source bodies, large earlier checkpoints and detailed run records remain local. Small learned circuits, procedure libraries and compact results are included in the repository. The 13-procedure retained artifact is a documented combination of previously acquired compatible procedures. Published measurements describe the recorded runs and separately labeled post-combination checks. The mathematical analysis and development plan do not add training evidence.